Complete System Design Notes for Beginners

Table of Contents

What is System Design?

System Design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specific requirements. It's about creating blueprints for large-scale distributed systems.

Key Goals:

- **Scalability**: Handle increasing load
- **Reliability**: System works consistently
- **Availability**: System is accessible when needed
- **Performance**: Fast response times
- **Maintainability**: Easy to update and modify

Why is it Important?

- Build systems that can handle millions of users
- Ensure systems don't crash under heavy load
- Create cost-effective solutions
- Plan for future growth

Scalability

Scalability is the ability of a system to handle increased load by adding resources.

Types of Scalability:

1. Vertical Scaling (Scale Up)

- Adding more power to existing machines
- Increase CPU, RAM, or storage
- **Pros**: Simple to implement
- **Cons**: Hardware limits, single point of failure

2. Horizontal Scaling (Scale Out)

- Adding more machines to the system
- Distribute load across multiple servers
- **Pros**: No hardware limits, fault tolerant
- **Cons**: More complex to implement

When to Scale?

- Response time increases
- Error rates go up
- Resource utilization is high (CPU, memory, disk)

Scaling Strategies:

- **Load balancing**: Distribute requests across servers
- **Caching**: Store frequently accessed data
- **Database optimization**: Indexing, query optimization
- **Content delivery**: Use CDNs for static content

Load Balancing

Load Balancing distributes incoming requests across multiple servers to ensure no single server becomes overwhelmed.

Types of Load Balancers:

1. Layer 4 Load Balancer (Transport Layer)

- Routes based on IP and port
- Faster but less intelligent
- Example: Network Load Balancer

2. Layer 7 Load Balancer (Application Layer)

- Routes based on content (HTTP headers, URLs)
- More intelligent routing
- Example: Application Load Balancer

Load Balancing Algorithms:

1. Round Robin

- Requests sent to servers in circular order
- Simple but doesn't consider server capacity

2. Weighted Round Robin

- Servers assigned weights based on capacity
- More powerful servers get more requests

3. Least Connections

- Route to server with fewest active connections
- Good for long-running requests

4. IP Hash

- Hash client IP to determine server
- Ensures same client goes to same server

Benefits:

- Improved response time
- Increased availability
- Better resource utilization
- Fault tolerance

Caching

Caching stores frequently accessed data in fast storage to reduce response time and database load.

Types of Caching:

1. Browser Cache

- Stores static files (CSS, JS, images) locally
- Reduces network requests

2. CDN Cache

- Geographically distributed cache
- Stores static content closer to users

3. Application Cache

- In-memory cache within application
- Examples: Redis, Memcached

4. Database Cache

- Query result caching
- Reduces database load

Caching Strategies:

1. Cache-Aside (Lazy Loading)

- Application manages cache
- Check cache first, then database
- Update cache when data changes

2. Write-Through

- Write to cache and database simultaneously
- Ensures data consistency
- Higher write latency

3. Write-Behind (Write-Back)

- Write to cache first, database later
- Better write performance
- Risk of data loss

Cache Considerations:

- **Cache Hit Ratio**: Percentage of requests served from cache
- **TTL (Time To Live)**: How long data stays in cache
- **Cache Invalidation**: Removing stale data
- **Cache Warming**: Pre-loading cache with data

Database Design

SQL vs NoSQL

SQL Databases (Relational)

- **Structure**: Tables with rows and columns
- **Schema**: Fixed structure
- **Examples**: MySQL, PostgreSQL, Oracle
- **ACID Properties**: Atomicity, Consistency, Isolation, Durability
- **Use Cases**: Financial systems, traditional applications

NoSQL Databases (Non-Relational)

- **Structure**: Various (document, key-value, graph, column-family)
- **Schema**: Flexible or schema-less
- **Examples**: MongoDB, Cassandra, DynamoDB
- **Use Cases**: Big data, real-time analytics, content management

Database Optimization:

1. Indexing

- Data structures that improve query speed
- Trade-off: Faster reads, slower writes
- Types: B-tree, Hash, Bitmap

2. Query Optimization

- Use efficient queries
- Avoid SELECT *
- Use appropriate joins
- Analyze execution plans

3. Database Partitioning

- Split large tables into smaller pieces
- Horizontal partitioning (sharding)
- Vertical partitioning (splitting columns)

Microservices vs Monolithic Architecture

Monolithic Architecture

- Single deployable unit
- All components tightly coupled
- **Pros**: Simple to develop, test, deploy initially
- **Cons**: Difficult to scale, technology lock-in

Microservices Architecture

- Application split into small, independent services
- Each service has its own database
- **Pros**: Independent scaling, technology diversity, fault isolation
- **Cons**: Complexity, network latency, data consistency challenges

When to Use Microservices:

- Large, complex applications

- Multiple teams working on different features
- Need to scale different components independently
- Want to use different technologies

Communication Between Services:

- **Synchronous**: HTTP/REST, gRPC
- **Asynchronous**: Message queues, event streaming

Message Queues

Message Queues enable asynchronous communication between services.

Key Concepts:

1. Producer

- Creates and sends messages
- Doesn't wait for response

2. Consumer

- Receives and processes messages
- Can have multiple consumers

3. Queue

- Stores messages temporarily
- FIFO (First In, First Out) typically

Types of Message Systems:

1. Point-to-Point

- One producer, one consumer

- Message consumed once
- Example: Task queues

2. Publish-Subscribe

- One producer, multiple consumers
- Message copied to all subscribers
- Example: Event notifications

Popular Message Queue Systems:

- **RabbitMQ**: Feature-rich, supports multiple protocols
- **Apache Kafka**: High-throughput, distributed streaming
- **Amazon SQS**: Managed service, highly scalable
- **Redis Pub/Sub**: In-memory, fast but not persistent

Benefits:

- Decoupling of services
- Asynchronous processing
- Better fault tolerance
- Load leveling

Content Delivery Networks (CDN)

CDN is a geographically distributed network of servers that deliver content to users from the closest location.

How CDN Works:

1. User requests content
2. CDN routes to nearest server

3. If content exists, serve from CDN
4. If not, fetch from origin server
5. Cache content for future requests

Types of Content:

- **Static**: Images, CSS, JavaScript, videos
- **Dynamic**: Personalized content (with edge computing)

CDN Benefits:

- **Reduced Latency**: Content served from nearby servers
- **Improved Performance**: Faster load times
- **Reduced Bandwidth**: Less load on origin servers
- **Better Availability**: Multiple servers reduce single point of failure

Popular CDN Providers:

- Cloudflare
- Amazon CloudFront
- Azure CDN
- Google Cloud CDN

Consistency and CAP Theorem

CAP Theorem

States that in a distributed system, you can only guarantee 2 out of 3:

1. Consistency

- All nodes see the same data simultaneously

- Read operations return the most recent write

2. Availability

- System remains operational
- Every request gets a response

3. Partition Tolerance

- System continues to work despite network failures
- Communication breakdowns between nodes

Trade-offs:

- **CA Systems**: Traditional databases (not distributed)
- **CP Systems**: Consistent but may be unavailable (MongoDB)
- **AP Systems**: Available but may be inconsistent (Cassandra)

Consistency Levels:

1. Strong Consistency

- All reads return most recent write
- Higher latency

2. Eventual Consistency

- System will become consistent over time
- Lower latency, higher availability

3. Weak Consistency

- No guarantees about when data will be consistent
- Highest performance

Replication and Sharding

Database Replication

1. Master-Slave Replication

- One master (writes), multiple slaves (reads)
- Master sends updates to slaves
- **Pros**: Read scalability, backup
- **Cons**: Single point of failure for writes

2. Master-Master Replication

- Multiple masters can accept writes
- More complex conflict resolution
- **Pros**: Write scalability, no single point of failure
- **Cons**: Consistency challenges

Database Sharding

Sharding splits data across multiple databases based on some criteria.

Sharding Strategies:

1. Horizontal Sharding

- Split rows across databases
- Example: User data by user ID ranges

2. Vertical Sharding

- Split columns across databases
- Example: User profile vs user activity data

3. Functional Sharding

- Split by feature/service
- Example: User data vs order data

Sharding Challenges:

- **Rebalancing**: Moving data when adding/removing shards
- **Cross-shard queries**: Queries spanning multiple shards
- **Hotspots**: Uneven data distribution

API Design

API (Application Programming Interface) defines how different software components communicate.

REST API Design

HTTP Methods:

- **GET**: Retrieve data
- **POST**: Create new resource
- **PUT**: Update entire resource
- **PATCH**: Update partial resource
- **DELETE**: Remove resource

Status Codes:

- **200**: Success
- **201**: Created
- **400**: Bad Request
- **401**: Unauthorized
- **404**: Not Found
- **500**: Internal Server Error

API Design Best Practices:

1. Use Nouns for Resources

- `/users` instead of `/getUsers`

- `/users/123` for specific user

2. Use HTTP Methods Correctly

- GET for reading, POST for creating
- PUT for updating, DELETE for removing

3. Version Your APIs

- `/v1/users` or `/v2/users`
- Maintain backward compatibility

4. Use Consistent Naming

- snake_case or camelCase consistently
- Plural nouns for collections

5. Implement Proper Error Handling

- Meaningful error messages
- Consistent error format
- Appropriate status codes

API Security:

- **Authentication**: Who is the user?
- **Authorization**: What can they do?
- **Rate Limiting**: Prevent abuse
- **HTTPS**: Encrypted communication

Security in System Design

Security Principles:

1. Defense in Depth

- Multiple layers of security
- If one layer fails, others protect

2. Least Privilege

- Give minimum necessary permissions
- Regularly review and revoke unused access

3. Fail Secure

- When system fails, default to secure state
- Don't expose sensitive information in errors

Common Security Measures:

1. Authentication & Authorization

- **Authentication**: Username/password, OAuth, JWT
- **Authorization**: Role-based access control (RBAC)

2. Data Protection

- **Encryption at Rest**: Encrypt stored data
- **Encryption in Transit**: Use HTTPS/TLS
- **Data Masking**: Hide sensitive data in logs

3. Input Validation

- Validate all user inputs
- Prevent SQL injection, XSS attacks
- Use parameterized queries

4. Rate Limiting

- Prevent abuse and DoS attacks
- Limit requests per user/IP
- Use techniques like token bucket

Security in Different Layers:

- **Network**: Firewalls, VPNs
- **Application**: Input validation, secure coding
- **Database**: Access controls, encryption
- **Infrastructure**: Security groups, IAM

Monitoring and Logging

Monitoring

Monitoring tracks system health and performance in real-time.

Types of Monitoring:

1. Infrastructure Monitoring

- CPU, memory, disk usage
- Network performance
- Server health

2. Application Monitoring

- Response times
- Error rates
- User activity

3. Business Monitoring

- Revenue metrics
- User engagement
- Conversion rates

Key Metrics:

1. Golden Signals

- **Latency**: Time to process requests
- **Traffic**: Number of requests
- **Errors**: Rate of failed requests
- **Saturation**: Resource utilization

2. SLI/SLO/SLA

- **SLI (Service Level Indicator)**: Actual measurements
- **SLO (Service Level Objective)**: Target values
- **SLA (Service Level Agreement)**: Contracts with users

Logging

Logging records events for debugging and analysis.

Log Levels:

- **DEBUG**: Detailed information for debugging
- **INFO**: General information
- **WARN**: Potential issues
- **ERROR**: Error conditions
- **FATAL**: Critical errors

Centralized Logging:

- Collect logs from all services
- Examples: ELK Stack (Elasticsearch, Logstash, Kibana)
- Structured logging (JSON format)

Alerting:

- Set up alerts for critical metrics
- Define escalation procedures
- Avoid alert fatigue

System Design Process

Step 1: Understand Requirements

Functional Requirements

- What should the system do?
- Core features and capabilities
- User interactions

Non-Functional Requirements

- **Performance**: Response time, throughput
- **Scalability**: Number of users, requests
- **Reliability**: Uptime requirements
- **Consistency**: Data consistency needs

Step 2: Estimate Scale

Calculate:

- Daily Active Users (DAU)
- Requests per second
- Data storage requirements
- Bandwidth needs

Example Calculations:

- 1 million DAU
- Each user makes 10 requests/day
- Total: 10 million requests/day
- Peak: 115 requests/second (assuming 10x peak)

Step 3: High-Level Design

Core Components:

- Client applications
- Load balancers
- Application servers
- Databases
- Caches
- Message queues

Draw System Architecture:

- Show data flow
- Identify main components
- Show connections between components

Step 4: Detailed Design

Focus on:

- Database schema
- API design
- Algorithm choices
- Data structures
- Caching strategy

Step 5: Address Scalability

Identify Bottlenecks:

- Database limitations
- Single points of failure
- Network bandwidth
- Storage constraints

Solutions:

- Database sharding
- Read replicas
- Caching layers
- CDN for static content

Step 6: Consider Trade-offs

Common Trade-offs:

- **Consistency vs Availability**
- **Performance vs Consistency**
- **Cost vs Performance**
- **Complexity vs Maintainability**

Step 7: Security and Monitoring

Security:

- Authentication/authorization
- Data encryption
- Rate limiting
- Input validation

Monitoring:

- Metrics collection
- Alerting
- Logging
- Health checks

Common System Design Patterns

1. Database per Service

- Each microservice has its own database
- Ensures loose coupling
- Data consistency challenges

2. API Gateway

- Single entry point for all client requests
- Handles authentication, rate limiting, routing
- Simplifies client interactions

3. Circuit Breaker

- Prevents cascading failures
- Fails fast when service is down
- Allows system to recover

4. Event Sourcing

- Store events instead of current state
- Can replay events to rebuild state
- Good for audit trails

5. CQRS (Command Query Responsibility Segregation)

- Separate read and write models
- Optimize each for its specific use case
- Better performance and scalability

Summary

System design is about creating robust, scalable, and maintainable systems. Key principles include:

1. **Start Simple**: Begin with basic architecture and add complexity as needed
2. **Think About Scale**: Design for current needs but plan for growth
3. **Consider Trade-offs**: Every decision has pros and cons
4. **Focus on Reliability**: Systems should be fault-tolerant
5. **Monitor Everything**: You can't fix what you can't measure
6. **Security First**: Build security into every layer
7. **Document Decisions**: Explain why you made certain choices

Remember: There's no single "correct" design. The best design depends on your specific requirements, constraints, and trade-offs. Practice by designing systems you use daily (like social media, e-commerce, or messaging apps) and gradually tackle more complex scenarios.

Practice makes perfect! Start with simple systems and gradually work your way up to more complex distributed systems. Good luck with your system design journey!